

Recursos ISIS-2211

HttpResource y HttpClient

Teoría

Dos herramientas, un rol cada una. Una le ofrece el estado reactivo, la otra le da control total sobre las acciones. Saber cuándo usar cada cual es lo que separa un componente que funciona de uno que escala.

Para leer (GET), `httpResource`

Carga datos del servidor y le entrega `value()`, `isLoading()` y `error()` listos para usar en la plantilla.

Para actuar (POST / PUT / DELETE), `HttpClient`

Ejecuta cualquier método HTTP. Usted maneja cuándo se dispara y qué pasa con la respuesta. Imprescindible para crear, editar y borrar.

El punto de partida

Ya tiene sus interfaces TypeScript, ya tiene un arreglo de prueba en el componente. Pero la realidad es que los datos viven en un servidor (un Gist de GitHub hoy, un back en Spring más adelante). Hay que ir a buscarlos.

Sin Angular, haría algo así con el API nativa del navegador:

sin-framework.ts

```
// Usted gestiona TODO: carga, error, datos
let films = [];
let isLoading = false;
let error = null;

fetch('data/films.json')
  .then(r => r.json())
  .then(data => { films = data; isLoading = false; })
  .catch(err => { error = err; isLoading = false; });
// Problema: ¿cómo aviso a la vista que cambió el estado?
```

⚠ El problema

Cada vez que cambia `films`, `isLoading` o `error`, Angular no sabe que debe repintar la pantalla. Tiene que conectar el mundo asíncrono de la red con el estado reactivo de la vista. Angular tiene dos herramientas para eso.

La estructura de datos que se usa en todos los ejemplos es la del archivo **films.json** del proyecto FilmSignals. Así se ve un elemento:

films.json (fragmento)

```
{
  "title":      "Interstellar",
  "year":       "2014",
  "director":   "Christopher Nolan",
  "imdbRating": "8.6",
  "imdbID":     "tt0816692",
  "poster":     "https://..."
}
```

Y la interfaz TypeScript que lo modela, el contrato que se firma con el compilador:

film.ts

```
export interface Film {  
  title:      string;  
  year:       string;  
  director:   string;  
  imdbRating: string;  
  imdbID:    string;  
  poster:    string;  
}
```

httpResource: GET reactivo sin esfuerzo

httpResource es una función de Angular (disponible desde v19) que hace el GET y le entrega el resultado ya empaquetado en signals. No hay subscribe, no hay manejo manual de estados.

La analogía

Imagine que contrata un mensajero con un letrero en su puerta. Le dice la dirección una sola vez. Él va, trae los datos, actualiza el letrero con el resultado (o con el error, si algo salió mal), y usted solo lee el letrero. Eso es `httpResource`: el mensajero con letrero incluido.

Anatomía mínima

Esto es todo lo que necesita para traer la lista de películas y conectarla a su plantilla:

film.service.ts

```
import { Injectable, inject } from '@angular/core';
import { httpResource } from '@angular/common/http';
import { Film } from './film';

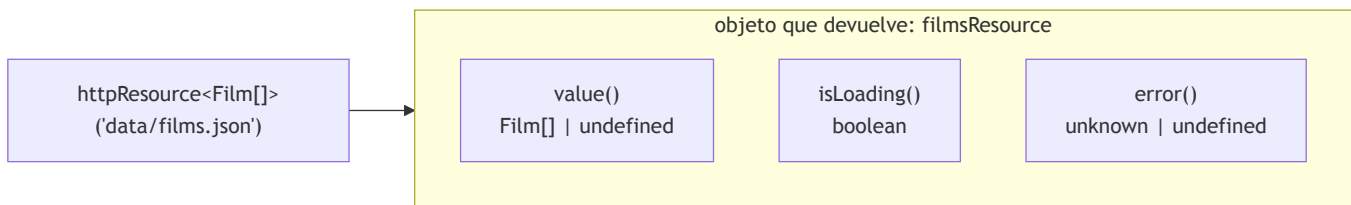
@Injectable({ providedIn: 'root' })
export class FilmService {

  // Una sola línea: URL → signals listas para usar
  filmsResource = httpResource<Film[]>('data/films.json');

}
```

❗ ¿Y dónde está el signal()?

No lo va a encontrar: `filmsResource` no es una signal, es un objeto con varias signals adentro (`value`, `isLoading`, `error`). `httpResource` las declara **por dentro**, igual que `computed()` tampoco le exige escribir `signal()` para funcionar.



❗ Nota

Recuerde declarar `provideHttpClient()` en su `app.config.ts` para que Angular tenga acceso al cliente HTTP. Sin esa línea verá un `NullInjectorError`.

Los tres signals que le regala

httpResource devuelve un objeto con varios signals. Los tres que usará siempre en la plantilla son:

Signal	Tipo	¿Qué contiene?	Cuándo usarlo
.value()	Film[] undefined	Los datos cuando ya llegaron. undefined mientras carga.	Pintar la lista con @for
.isLoading()	boolean	true mientras la petición está en vuelo.	Mostrar un spinner o texto "Cargando..."
.error()	unknown undefined	El error si la petición falló. undefined en caso de éxito.	Mostrar un mensaje de error con @if

Cómo se ven en la plantilla

film-list.component.html

```
<!-- Mientras carga -->
@if (filmService.filmsResource.isLoading()) {
  <p>Cargando películas...</p>
}

<!-- Si hubo error -->
@if (filmService.filmsResource.error()) {
  <p class="error">Error al cargar. Intente de nuevo.</p>
}

<!-- Los datos cuando llegan -->
@for (film of filmService.filmsResource.value() ?? []; track film.imdbID)
{
  <div class="card">{{ film.title }}</div>
}
```

📘 Conexión con FilmSignals

En el ejemplo **FilmService** del proyecto, el mismo resultado se logra con cuatro signals manuales (`filmsListSignal`, `loadingSignal`, `errorSignal`, `selectedFilmSignal`) y los métodos que los actualizan dentro de `tap()`. `httpResource` empaqueta ese patrón en una línea para el caso más común: traer una lista y mostrarla.

Demo interactiva: los tres estados en acción

Simule lo que Angular haría al ejecutar `httpResource('data/films.json')`:

`isLoading() : false`

`value() : undefined`

`error() : undefined`

▶ Simular carga exitosa

× Simular error de red

↺ Reiniciar

Pulse un botón para ver cómo cambian los signals.

HttpClient: cuando necesita actuar

`httpResource` es perfecta para leer. Pero si el usuario quiere **crear una película nueva, editarla o borrarla**, necesita enviar datos al servidor, y para eso existe **HttpClient**.

⚠ La regla mnemotécnica

resource para leer, client para actuar.

GET automático → `httpResource`.

POST, PUT, DELETE → `HttpClient`.

¿Cuándo usar cada uno? Tabla CRUD

GET, httpResource

Leer la lista completa. Angular gestiona carga y error automáticamente.

POST, HttpClient

Enviar un nuevo objeto al servidor. Usted define qué pasa al recibir la respuesta.

PUT, HttpClient

Reemplazar un objeto existente por su id.

DELETE, HttpClient

Eliminar un recurso del servidor por su id.

Anatomía de un POST, paso a paso

film.service.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient }          from '@angular/common/http';
import { Observable }          from 'rxjs';
import { Film }                from './film';

@Injectable({ providedIn: 'root' })
export class FilmService {

  // 1. Inyecta HttpClient, igual que en la sesión 2 de R1
  private http: HttpClient = inject(HttpClient);
  private apiUrl: string = 'https://mi-api.com/films';

  // 2. El método devuelve un Observable, no ejecuta nada todavía
  createFilm(newFilm: Film): Observable<Film> {
    return this.http.post<Film>(this.apiUrl, newFilm);
  }
}
```

⚠ El genérico <Film> no es opcional

Si escribe `this.http.post(this.apiUrl, newFilm)` sin el `<Film>`, Angular asume que la respuesta es de tipo `Object`. El backend sigue devolviendo el `Film` completo (con su `id` ya asignado) — el dato llega igual por la red — pero TypeScript ya no sabe que ese `Object` tiene `title`, `id`, etc. En el componente, `creada.title` deja de compilar con un error como *“Property ‘title’ does not exist on type ‘Object’”*. Se siente como si “no llegara el objeto”, cuando en realidad el problema es de tipos, no de red.

film-form.component.ts

```
import { Component, inject }    from '@angular/core';
import { HttpResponse }         from '@angular/common/http';
import { FilmService }         from '../film.service';
import { Film }                from '../film';

export class FilmFormComponent {

  private filmService: FilmService = inject(FilmService);

  onSubmit(formData: Film): void {

    // 3. subscribe() es quien activa la petición HTTP
    this.filmService.createFilm(formData).subscribe({
      next: (creada: Film) => {
        // 4a. Éxito: creada es un Film real, con sus propios atributos.
        // Por eso puede leer creada.imdbID: no es un objeto genérico.
        console.log('Película creada, imdbID asignado:', creada.imdbID);
      },
      error: (err: HttpResponse) => {
        // 4b. Error: algo salió mal en el servidor
        console.error('Error al crear:', err);
      }
    });
  }
}
```

❗ Punto clave, Observable frío

`http.post()` construye la petición pero **no la envía**. El Observable es “frío”: solo se dispara cuando alguien llama `.subscribe()`. Esto es diferente a `httpResource`, que se suscribe solo.

El ciclo completo de una acción

flujo mental

Usuario hace clic en "Guardar"

↓

Componente llama `filmService.createFilm(datos)`

↓

Servicio llama `http.post(url, datos)` ← Observable creado, no disparado

↓

Componente llama `.subscribe()` ← Aquí se envía la petición

↓

Servidor procesa y responde

↓

`next()` o `error()` se ejecutan

↓

Componente actualiza el estado (signal) → vista reacciona

Demo interactiva: simulador de POST

Rellene los campos y observe qué JSON se enviaría al servidor y qué respuesta llegaría:

TITLE

Nombre de la película

YEAR

2024

DIRECTOR

Nombre del director

IMDBRATING

8.5

```
{
  "title": "",
  "year": "",
  "director": "",
  "imdbRating": ""
}
```

[↑ Enviar POST](#)[↶ Limpiar](#)

La respuesta del servidor aparecerá aquí...

Recurso interactivo

Visualización: cómo funciona `http.post()` en Angular [→](#)

Recorrido interactivo paso a paso del flujo completo: componente, servicio, Observable y `subscribe()`.

PUT y DELETE: la misma idea, distinto método

[Ver snippet de PUT](#)[Ver snippet de DELETE](#)

El patrón del servicio completo

En la práctica, un servicio Angular que maneja una entidad usa los dos: `httpResource` para la lista (siempre actualizada) y `HttpClient` para las operaciones que modifican datos.

film.service.ts, el patrón completo

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, httpResource } from '@angular/common/http';
import { Observable }          from 'rxjs';
import { Film }                from './film';

@Injectable({ providedIn: 'root' })
export class FilmService {

  private http: HttpClient = inject(HttpClient);
  private apiUrl: string = 'api/films';

  // La lista, siempre reactiva (GET)
  filmsResource = httpResource<Film[]>(this.apiUrl);
  // En la plantilla:
  // filmsResource.value() → los datos
  // filmsResource.isLoading() → spinner
  // filmsResource.error() → mensaje

  // Crear
  create(f: Film): Observable<Film> {
    return this.http.post<Film>(this.apiUrl, f);
  }

  // Editar
  update(id: string, f: Film): Observable<Film> {
    return this.http.put<Film>(`${this.apiUrl}/${id}`, f);
  }

  // Borrar
  remove(id: string): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Conexión con FilmSignals

En el `FilmService` del ejemplo, los signals se manejan manualmente porque el proyecto usa un JSON local y quiere controlar cada estado por separado (incluyendo `selectedFilmSignal`, que no existe en `httpResource`). Cuando conecte su app a un back real en Spring, el patrón de arriba es el camino natural: `httpResource` para el GET, `HttpClient` para todo lo demás.

Refrescar un `httpResource` después de escribir, `.reload()`

Después de crear, editar o borrar algo con `HttpClient`, la lista que trajo `httpResource` queda **desactualizada**: sigue mostrando lo que había antes de la escritura. `httpResource` resuelve esto con un método propio, `.reload()`, que vuelve a disparar el mismo GET y actualiza `value()` con la respuesta nueva.

film-form.component.ts

```
onSubmit(formData: Film): void {
  this.filmService.createFilm(formData).subscribe({
    next: () => {
      // El POST ya terminó: la lista que muestra otro componente
      // sigue mostrando los datos viejos hasta que se refresque.
      this.filmService.filmsResource.reload();
    },
    error: (err: HttpErrorResponse) => console.error('Error al crear:',
err),
  });
}
```

Por qué esto sincroniza componentes que no se conocen

Si `filmsResource` vive en un servicio `providedIn: 'root'` (como el singleton que vio en [Angular Signals](#)), todo componente que inyecte ese servicio comparte exactamente el mismo objeto `filmsResource`. Cuando uno de ellos llama `.reload()`, el `value()` que lee **cualquier otro** componente se actualiza también, sin que hagan falta `@Input/@Output` ni que un componente sepa que el otro existe: ambos están mirando el mismo recurso.

Una regla más para recordar

regla de oro

```
// ✓ El servicio es el único que habla con la red
// ✓ El componente llama métodos del servicio
// × NUNCA inyecte HttpClient directamente en un componente

// La presentación no sabe de dónde vienen los datos.
// Si mañana cambia el back, solo se toca el servicio.
```

Autoevaluación

1. Quiere mostrar la lista de películas en su componente y que se pinte automáticamente cuando lleguen los datos. ¿Cuál es la herramienta más adecuada?

- ☐ HttpClient con `.subscribe()`
- ☐ `httpResource(url)`
- ☐ `fetch()` del navegador
- ☐ `ngOnInit()` directamente

2. Un estudiante escribe `this.http.post('api/films', newFilm)` en su servicio y nada ocurre

En el siguiente código `httpClient.post<Film>('http://localhost:3000/films', { title: 'El Padrino' })`, `httpClient` es un servicio que se encuentra en la red. ¿Qué falta para que la petición HTTP se ejecute?

- ☐ Falta el tipo genérico `<Film>`
- ☐ Falta llamar `.subscribe()`
- ☐ Falta el `await`
- ☐ Falta `import` de `HttpClient`

3. ¿Cuál de estos signals NO proporciona `httpResource` automáticamente?

- ☐ `value()`
- ☐ `isLoading()`
- ☐ `selectedItem()`
- ☐ `error()`

4. En `FilmService` del proyecto `FilmSignals`, los signals internos son `private` (por ejemplo `filmsListSignal`) y el servicio expone getters públicos para leerlos. ¿Por qué?

- ☐ Para mejorar el rendimiento de Angular
- ☐ Para que ningún componente llame `.set()` directamente y solo el servicio controle el estado
- ☐ Porque TypeScript lo exige en servicios inyectables
- ☐ Para que funcione con `httpResource`

5. Su componente necesita borrar una película cuando el usuario pulsa un botón. ¿Cuál es el flujo correcto?

- ☐ Usar httpResource con método DELETE en la URL
- ☐ Inyectar HttpClient en el componente y llamar .delete() directo
- ☐ El componente llama filmService.deleteFilm(id) y el servicio usa http.delete().subscribe()
- ☐ Modificar directamente el array en el componente con splice()

6. Un servicio providedIn: 'root' tiene un solo filmsResource. Dos componentes distintos lo inyectan. Uno de ellos crea una película y llama filmsResource.reload(). ¿Qué pasa con el otro componente?

- ☐ No pasa nada, cada componente tiene su propia copia del resource
- ☐ También ve la lista actualizada, porque ambos comparten el mismo objeto filmsResource
- ☐ Hay que notificarlo manualmente con un @Output

Comprobar